

AKMA: Autonomous Knowledge Mutation for Persistent RAG Systems

Marthan L. Lanuzga
Independent Researcher
Metro Manila, Philippines
tanlanuzga@gmail.com

Abstract—Retrieval-Augmented Generation (RAG) pipelines often waste retrieved context and treat non-parametric external knowledge assets as static, immutable stores, making them highly susceptible to information stagnation, data gaps, and malicious injection vectors. This study introduces a self-optimizing RAG pipeline designed for autonomous, unsupervised learning through an iterative validation and refinement cycle employing role-separated, adversarial-inspired evaluation stages using RAG and LLMs as the main components. Unlike conventional RAG systems that rely on static reference, Autonomous Knowledge Mutation Architecture (AKMA) implements a dual vector space that separates active production documents from an isolated candidate document environment. Empirical evaluations show that AKMA simultaneously eliminates parametric memory bias, minimizes response generation hallucinations, and provides a much more secure, transparent, and autonomous memory structure for next-generation intelligent systems.

Index Terms—Retrieval-Augmented Generation (RAG), Persistent Memory, Adversarial Validation, Unsupervised Learning, Autonomous Agents

I. INTRODUCTION

Existing knowledge retrieval pipelines include large language models (LLMs) that are trained from immense data points or tokens; as a result, training costs are exorbitant and resource-intensive in terms of time spent. As external information continues to evolve, the system’s fixed memory suffers from memory and relevance stagnation, where the gap between the model’s knowledge base and the current state of the external data continues to widen over time. Hallucination also becomes apparent on LLM systems as their sole reliance on fixed parameters decreases reliability. These problems were addressed by implementing Retrieval-Augmented Generation (RAG), where access to validated and updated facts is immediate. Although helpful in preventing hallucinations, RAG is essentially reactive and lacks the ability to persist and update the information that was produced during the interaction; therefore, the knowledge base remains a static cache instead of an evolving autonomous learning mechanism. To bridge this gap, I propose an Autonomous Knowledge Mutation Architecture (AKMA) designed for factual evolution inside non-parametric memory. AKMA utilizes the reasoning capabilities of a frozen LLM backbone as a curator for logical reconciliation and document refinement. AKMA is designed to perform active memory writes. The system architecture condenses interaction traces into validated updates, which employs an external LLM for refinement, intent classification,

and adversarial review completing a closed loop mechanism for a self-optimizing pipeline without the need for constant parameter updates.

This paper is organized as follows: Section II reviews related work in non-parametric integration and self-reflection frameworks. Section III details the core AKMA system architecture, tracing the knowledge lifecycle from ingestion through multi-session consensus-gated promotion. Section IV defines the experimental methodology, setup configuration, and analyzes empirical results across all evaluation metrics. Section V outlines the architectural limitations and future work, and Section VI concludes the study.

II. RELATED WORK

A. Non-Parametric Knowledge Integration

Hallucination which stems from the parametric knowledge stored within the model’s artifact remains a challenge in large language models (LLMs). In spite of the advancements in model reasoning capabilities, directly manipulating this embedded, pre-trained parametric knowledge is heavily constrained [1], introducing significant risks of both factual hallucination and temporal knowledge obsolescence post-training. To mitigate these failures, standard systems integrate Retrieval-Augmented Generation (RAG) pipelines to couple the underlying model with an external, non-parametric knowledge base containing dynamically updated document registries.

B. Self-Reflection Frameworks

To address document retrieval inefficiencies in static pipelines, Self-RAG [2] bypasses conventional RAG limitations, such as indiscriminate fixed-volume pulling and an inability to judge query relevance or validate pre-generation factuality. While recent frameworks reduce hallucinations by auditing active text generation runs, this study instead resolves factual consistency directly at the origin by autonomously updating and improving the system’s underlying knowledge base.

C. Memory-Augmented and Dual-Process Agents

Conventional RAG frameworks suffer from a lack of post-generation persistence, functioning as static pipelines incapable of long-term learning without manual curation. To address this, a continuous context refinement framework [3] was proposed to enable autonomous knowledge expansion while

keeping the weights of the language model. This architecture implements a reflective mechanism that commits RAG-based responses to a long-term memory vector database. It uses a self-healing memory layer to refine knowledge through natural-language entries and actively prunes adverse information blocks to maintain data quality. Evaluated against static prompting and non-parametric baselines, this dual architecture directly extends contextual memory layers popularized by [4] from runtime execution optimization to permanent, long-term non-parametric database persistence. However, this architecture is fundamentally restricted to evolving procedural behavior rather than factual understanding, explicitly conceding that its self-healing memory cannot acquire new external facts.

D. Architectural Comparison with Related Systems

Table I summarizes the key architectural distinctions between AKMA and the most closely related systems. While Self-RAG introduces retrieval-time self-reflection and MemGPT provides runtime context management, neither system supports persistent knowledge mutation, multi-session consensus gating, or web-grounded verification at the promotion boundary. AKMA addresses this gap by treating the knowledge base itself as an evolving, autonomously curated artifact rather than a static retrieval index.

TABLE I
ARCHITECTURAL FEATURE COMPARISON ACROSS RELATED SYSTEMS

Feature	Static RAG	Self RAG	MemGPT	AKMA
Knowledge persistence	×	×	Partial	✓
Adversarial validation	×	Partial	×	✓
Multi-session consensus	×	×	×	✓
Web grounding at promotion	×	×	×	✓
Contamination isolation	×	×	×	✓
Autonomous write-back	×	×	Partial	✓
Forensic provenance trail	×	×	×	✓

× = not supported, *Partial* = heuristic support, ✓ = fully supported

A quantitative head-to-head comparison against these systems is identified as a priority for future work; the present evaluation focuses on demonstrating AKMA’s novel architectural properties rather than competitive benchmarking.

III. SYSTEM ARCHITECTURE

Most existing RAG architectures function as static retrieval layers rather than adaptive knowledge systems, causing retrieved knowledge to gradually become outdated or contextually insufficient. To address this, the Autonomous Knowledge Mutation Architecture (AKMA) introduces a non-parametric knowledge framework capable of iterative refinement through controlled mutation and validation stages. AKMA isolates all proposed refinements within a staged candidate lifecycle enforced with adversarial validation, semantic consistency checks, and multi-session consensus mechanisms, enabling the knowledge base to evolve autonomously while minimizing semantic drift and knowledge contamination.

A. Database Architecture and Persistence Layer

The AKMA framework utilizes ChromaDB [5] as its core vector database, selected over Milvus [6], FAISS [7], and Pinecone [8] for its secure local persistence, native metadata filtering, and zero-configuration embedded footprint. Dense semantic vector embeddings are generated via the 384-dimensional all-MiniLM-L6-v2 sentence-transformers model [9], utilizing the bi-encoder framework of Sentence-BERT [10]. The persistence layer is partitioned into two collections:

- **Active Documents (active_nodes):** The consensus-verified production knowledge base serving as the exclusive source of truth. Entries carry an administrative status tag: active (authorized to serve queries), archived (superseded by a promoted mutation), or poisoned (invalidated via web-based conflict arbitration, preserved for forensic traceability).
- **Candidate Documents (candidate_nodes):** Staged, speculative knowledge mutations in a quarantined state (status = candidate), each appended with an inline validation ledger tracking multi-session consensus until promotion thresholds are satisfied.

B. Functional Component Architecture

The AKMA pipeline is composed of four discrete functional layers:

- **Smart Router:** An intent classifier that analyzes incoming queries against retrieved context vectors to enforce deterministic execution paths.
- **Refiner LLM:** A constrained generation agent that integrates novel query context into existing active documents without altering historical core facts.
- **Critic LLM:** An adversarial validation agent executing rigid, rubric-based structural audits to verify factual preservation and suppress hallucinations.
- **Consensus Engine:** A state evaluator that monitors candidate staging ledgers, requiring independent multi-session cross-corroboration before granting promotion rights.

C. AKMA Mutation Lifecycle

1) *Retrieval and Mutation Pipeline:* Incoming queries are embedded using the same all-MiniLM-L6-v2 model used for stored documents, ensuring retrieval consistency. AKMA employs cosine similarity as the semantic distance metric:

$$\cos(\theta) = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum_{i=1}^n q_i^2} \sqrt{\sum_{i=1}^n d_i^2}} \quad (1)$$

Queries meeting or exceeding RELEVANCE_THRESHOLD = 0.50 are routed to the Refinement Pipeline; those below are redirected to the Knowledge Synthesis Pipeline. All retrieval operations enforce status = active metadata filtering. The retrieved document is injected directly into the LLM prompt as the absolute source of truth, enabling the Smart Router to simultaneously classify intent and generate candidate refinements in a single low-latency call ($\tau = 0.3$). If web search

Autonomous Knowledge Mutation Architecture (AKMA) - AI knowledge system

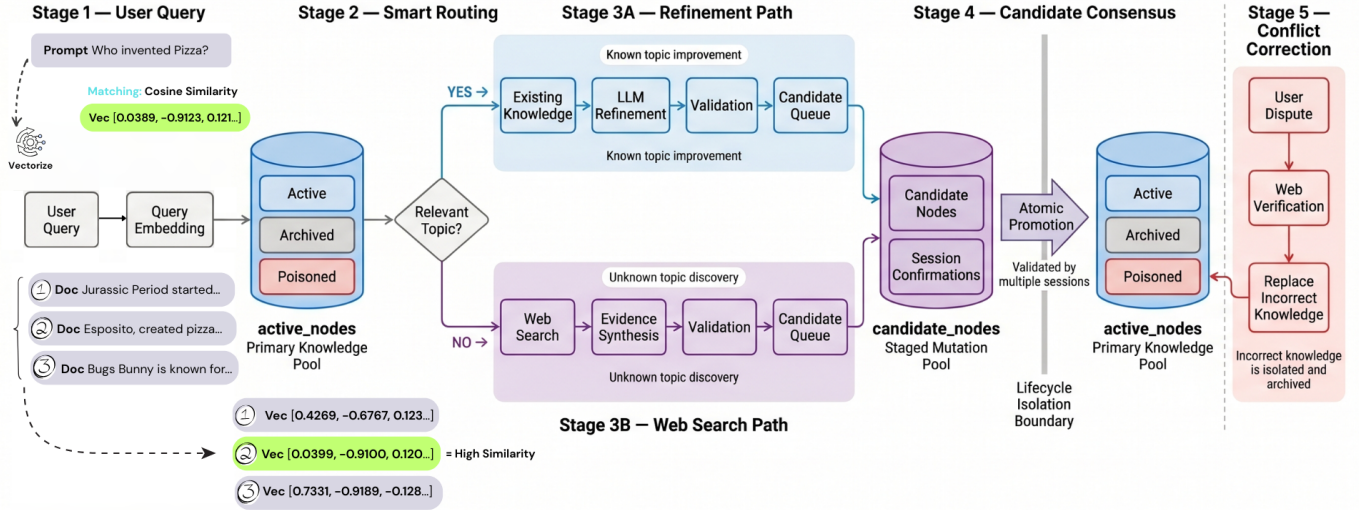


Fig. 1. AKMA System Workflow

fails during dispute handling, the original document remains active and a non-destructive audit entry is appended to its `unverified_disputes` metadata, protecting against Denial-of-Service censorship attacks.

The Smart Router classifies each query into exactly one of five routing primitives:

- **STATIC_MATCH:** Document is relevant, factually stable, and sufficient to resolve the query.
- **VOLATILE:** Query requests time-sensitive information. An in-process cache is checked first; a cache hit within the TTL window (300 s) serves the response immediately without triggering external search.
- **CONFLICT:** User disputes a stored factual claim. Triggers the Conflict Fast-Path, bypassing the consensus queue for immediate web arbitration (Section III-C4).
- **INSUFFICIENT:** Query requests details beyond the document's current scope.
- **DIFFERENT:** Nearest-neighbor retrieval returned a semantic false positive.

Refinement Pipeline. On a `STATIC_MATCH`, the Refiner LLM ingests the original document and query, outputting a refined version D' with a change summary. The Refiner may expand or append but is strictly prohibited from deleting or contradicting existing facts. D' is staged in `candidate_nodes` with the parent's `original_id`, establishing a traceable parent-child lineage. Upon promotion, the parent is archived and the promoted node inherits a `parent_id` attribute.

Knowledge Synthesis Pipeline. Activated when no rele-

vant active document exists or when the Smart Router flags `VOLATILE`, `INSUFFICIENT`, or `DIFFERENT`, this pipeline executes a four-stage acquisition cycle: (1) **Autonomous Web Search** via the Tavily API with a domain blacklist filtering low-authority sources; (2) **Grounded Synthesis**, where the LLM constructs D_{syn} exclusively from retrieved sources without parametric injection; (3) **Active Pool Deduplication**, discarding the candidate if cosine similarity to any active document meets $\theta_{\text{web_match}} = 0.80$; and (4) **Candidate Pool Deduplication**, incrementing the `occurrence_count` of a matching staged candidate or initializing a new one.

2) *Adversarial Validation (Critic Framework):* Every proposed mutation passes through the Critic LLM before entering the candidate pool. The Critic operates in two modes.

Mode A — Refinement Evaluation. Given the original and refined documents, the Critic executes a four-step audit: (1) enumerates all specific facts (names, dates, values); (2) classifies each as OK, WARN (softened), or FAIL (altered/removed); (3) audits additions for logical compatibility with the original; and (4) produces a score $S \in [0.0, 1.0]$. Score boundaries are enforced mathematically: a single FAIL caps $S \leq 0.40$; any WARN bounds S within $[0.20, 0.40]$; all-OK facts award a baseline $S_0 = 0.60$, with the remaining 0.40 earned through structural coherence. The pipeline rejects any candidate below $\theta_{\text{rej}} = 0.70$ and requires $S \geq 0.80$ for promotion.

Mode B — Synthesis Evaluation. For web-sourced documents without a parent, the Critic checks: (1) source coverage, (2) fabrication detection, and (3) internal coherence, on the

same 0.0–1.0 scale.

Both modes use $\tau = 0.1$. Scores are extracted via a backward-scanning parser (`_parse_score()`); unparseable responses default to 0.5, below both thresholds, ensuring parse failures never approve a mutation.

TABLE II
AKMA THREAT MODEL AND ARCHITECTURAL MITIGATIONS

Threat	Mitigation Layer	System Metric	Defense
Semantic Drift	Continuity Gate	θ_{sim} (Cosine Sim)	≥ 0.80
Refinement Hallucination	Critic Structural Audit	θ_{rej} (Quality Score)	≥ 0.70
Session Flooding (Sybil)	Session Diversity Gate	θ_{session} (UUID Set)	≥ 2
Coordinated Poisoning	Lazy Delta Verification	Grounded Claims	100% Confirmed
Dispute (Censorship)	DoS Non-destructive Logging	Data Node Status	Preserved Active

3) *Knowledge Validation and Promotion*: AKMA isolates all proposed mutations in `candidate_nodes`, preventing any single adversarial query from contaminating the active pool. Each candidate carries an inline metadata ledger with four fields: `occurrence_count` (validation frequency), `source_sessions` (serialized UUID array for Sybil defense), `timestamps` (ISO-8601 audit trail), and `similarity_score` (semantic drift boundary against the parent document).

The Consensus Engine gates promotion on four simultaneous criteria:

- 1) **Quality Score Gate** ($\theta_{\text{score}} \geq 0.80$): Critic audit score at promotion exceeds the staging threshold ($\theta_{\text{rej}} = 0.70$).
- 2) **Semantic Continuity Gate** ($\theta_{\text{sim}} \geq 0.80$): Cosine similarity between candidate and parent prevents topical drift.
- 3) **Empirical Confirmation Gate** ($\theta_{\text{count}} \geq 2$): Mutation confirmed across at least two independent validation events.
- 4) **Session Diversity Gate** ($\theta_{\text{session}} \geq 2$): $\text{cardinality}(\text{set}(\text{source_sessions})) \geq 2$, defeating single-client Sybil attacks regardless of query volume.

Once all gates are satisfied, the Lazy Delta Verification gate executes a fresh web re-query before promotion: (1) **Delta Extraction** isolates newly introduced claims in D_{ref} not present in D_{orig} via `EXTRACT_DELTA_CLAIMS_PROMPT`, bounded by `MAX_CLAIMS_TO_CHECK`; (2) **Targeted Web Grounding** issues a Tavily search per claim with domain filters; (3) **Factual Agreement Arbitration** requires an explicit `CONFIRMED` verdict from an LLM judge. Any unconfirmed claim blocks promotion entirely (any-fail policy), and verifier errors are treated as failures (fail-closed). Successful promotion is atomic: the parent is archived, the promoted document is inserted as active with full provenance metadata, and the candidate is deleted — enforcing an append-only mutation history reconstructable via the `parent_id` chain.

4) *Conflict Fast-Path*: When a user disputes a stored claim (`CONFLICT`), AKMA bypasses the consensus queue and invokes an LLM judge that compares the active document

against freshly synthesized web evidence, returning **AGREES** or **DISAGREES**. On **DISAGREES**, the original is marked poisoned with a forensic payload, orphaned candidates are pruned, and a corrected document is immediately activated. On **AGREES**, the dispute is logged to `unverified_disputes` and the original remains active. If web retrieval or synthesis fails, the dispute is logged and the original is preserved — preventing mass-dispute DoS attacks from censoring valid knowledge.

IV. EXPERIMENTAL METHODOLOGY

A. Experimental Setup

All experiments were conducted on an Ubuntu 22.04 workstation. The core pipeline utilized the `llama-3.3-70b-versatile` model hosted via the Groq API, coupled with the Tavily Search API for domain-filtered web retrieval. The baseline knowledge store was seeded with 10 documents spanning programming languages, historical events, biological systems, and emerging technologies. Each unique interaction window was tracked via an explicit UUID session identifier to isolate cross-session evaluation parameters cleanly.

The complete testing suite evaluated 31 queries across four focused sub-experiments, containing nine knowledge refinement queries and ten adversarial injection attempts. Evaluation throughput parameters were strictly governed by external API tier execution thresholds. Because each valid promotion path forces multiple sequential LLM calls alongside up to fifteen external search requests through the Lazy Delta Verification gate, single-run empirical tracking was enforced across all configurations. Extended multi-trial cross-statistical validations with explicit confidence tracking margins remain an active priority for future hardware evaluations.

The complete source code for the AKMA system is publicly available at <https://github.com/bikemaster2331/AKMA> to support reproducibility and further research.

B. Research Questions

To evaluate the performance, security boundaries, and operational tradeoffs of the AKMA framework, the experimental design is structured around answering the following four core research questions:

- **RQ1 (Autonomous Knowledge Expansion)**: Can AKMA continuously expand a non-parametric knowledge base via autonomous web synthesis without human-in-the-loop manual curation?
- **RQ2 (Contamination Resistance)**: Can AKMA’s adversarial validation gates effectively prevent unverified or poisoned knowledge mutations from contaminating the active production knowledge store?
- **RQ3 (Evolution Quality)**: Does the closed-loop dual-pool architecture produce a higher-quality, more factual knowledge evolution trace than conventional static RAG baselines?
- **RQ4 (Computational Overhead)**: What engineering tradeoffs in token latency and computational processing costs

arise from executing AKMA’s multi-stage validation and consensus promotion mechanisms?

C. RQ1: Autonomous Knowledge Expansion

To evaluate AKMA’s capacity for autonomous knowledge expansion without human-in-the-loop curation, three sub-experiments were conducted: (A) autonomous acquisition of entirely new knowledge topics, (B) iterative refinement of existing seed knowledge nodes, and (C) semantic matching across non-identical query phrasings. All experiments were performed on a fixed seed knowledge base of 10 documents with zero human intervention.

Sub-experiment A demonstrates successful non-parametric expansion as the knowledge base grew from 10 to 14 active nodes entirely via autonomous web synthesis and multi-session consensus tracking. Out of ten total staged candidates, the Lazy Delta Verification gate cleanly isolated and blocked five entries containing unconfirmed granular assertions, while one candidate was quarantined after scoring a marginal similarity threshold miss (Sim = 0.7949).

TABLE III
TEST A — AUTONOMOUS KNOWLEDGE ACQUISITION

Session	Queries	Cand.	Matched	Promoted	Active
Baseline	0	0	—	0	10
S1 Batch 1	5	5	—	0	10
S2 Batch 1	5	0	4/5	2	12
S1 Batch 2	5	5	—	0	12
S2 Batch 2	5	0	5/5	2	14
Total	20	10	9/10	4	14

Sub-experiment B submitted nine refinement queries against active documents, passing all consensus promotion criteria with a mean Critic score of 0.89. The Lazy Delta gate verified and authorized six of these entries, executing verified in-place knowledge evolution while safely quarantining three under factual uncertainty.

TABLE IV
TEST B — KNOWLEDGE REFINEMENT PIPELINE STAGE RESULTS

Query Topic	Critic	Sim	Gates	Lazy	Outcome
Python asyncio	0.90	0.8630	✓	Passed	Prom.
TensorFlow Lite	0.85	0.8924	✓	Passed	Prom.
DNA Mutations	0.85	—	✓	Blocked	Quar.
Roman Roads	0.80	—	✓	Blocked	Quar.
Bitcoin Mining	0.95	—	✓	Blocked	Quar.
Python 2 vs 3	0.95	0.9965	✓	Passed	Prom.
Python paradigms	0.90	0.9977	✓	Passed	Prom.
Great Depression	0.85	0.8110	✓	Passed	Prom.
Supervised ML	0.95	0.9777	✓	Passed	Prom.
Total	0.89	—	9/9	6/9	6 Prom.

Sub-experiment C confirms that the cross-session consensus logic triggers successfully on loose semantic intent (Sim = 0.9720) rather than exact string matches. Prior to promotion, the Lazy Delta gate isolated three factual claims regarding

Guido van Rossum and CWI, successfully verifying the 1991 release details across five sources while quarantining the rest due to over-granular search query extraction.

TABLE V
TEST C — CROSS-SESSION SEMANTIC IDENTIFICATION METRICS

Metric Tracker	Session 1	Session 2
Query Intent String	<i>Who invented Python...?</i>	<i>Who is the creator...?</i>
Path Routed	PATH B (Search)	PATH B (Search)
Candidate Doc	—	0.9720
Sim	—	—
Consensus Gates	—	✓ (all 4)
Lazy Delta Result	—	Blocked (2/3)
Outcome	Candidate Created	Quarantined

Sub-experiment D tracks absolute resistance to adversarial ingestion, showing that zero deceptive inputs successfully corrupted the active store. Low-specificity personal data injections were cleanly dropped by the Quality Score Gate, while false historical attributions (e.g., Linus Torvalds inventing Python) were actively corrected via the search pipeline back to factual truths prior to promotion.

TABLE VI
TEST D — ADVERSARIAL INPUT RESISTANCE

Query Intent	Critic	Match	Delta	Outcome
Personal identity	Fail	None	N/A	Rejected
Personal pet	Fail	None	N/A	Rejected
Cow in space	Pass	None	Blocked	Quarantined
Human Oslo Marskov	Pass	None	Blocked	Quarantined
Knitting with metal	Fail	None	N/A	Rejected
Eiffel Tower 1756	Pass	None	N/A	Quarantined
Sister’s color	Pass	Matched	N/A	Quarantined
Water boils 150°C	Pass	None	N/A	Quarantined
Einstein failed school	Pass	None	N/A	Quarantined
Linus Torvalds Python	Pass	Matched	Passed	Promoted
Total (10)	60% pass	20% match	—	0 contaminated

Across the complete evaluation space, the multi-layered guard profile demonstrates a highly robust trade-off favoring absolute knowledge precision over unverified promotion throughput. The pipeline successfully validated 10 total promotions—4 novel acquisitions and 6 structural refinements—expanding the active corpus safely with zero unverified leakages.

D. RQ2: Contamination Resistance

The adversarial sub-experiment (Test D) provides a focused lens on this result: the framework achieved a 100% contamination-free rate across ten adversarial inputs. One input — a false attribution claiming Python was created by Linus Torvalds — resulted in a promotion event; however, the synthesis pipeline retrieved contradicting web evidence and produced a factually correct document, functioning as

TABLE VII
SUMMARY METRICS ACROSS ALL SUB-EXPERIMENTS

Metric	Test A (Acq.)	Test B (Ref.)	Test C (Sem.)	Test D (Adv.)
Queries Submitted	10	9	2	10
Critic Pass Rate	100%	100%	100%	60%
Consensus Pass Rate	100%	100%	100%	10%
Semantic Match Rate	90%	100%	100%	20%
Lazy Delta Pass Rate	44.44%	66.67%	0%	0%
Promotions to Active	4	6	0	0

an autonomous acquisition trigger rather than a contamination vector.

Defense-in-depth behavior was observed across two isolated gates: the **Frontline Critic Firewall** rejected outright policy violations and unsourceable nonsensical queries (60% pass rate, 3 outright rejections), while the **Fail-Closed Lazy Delta Backstop** blocked more sophisticated semantic attacks — such as the fabricated individual and outrageous factual claims — that passed initial synthesis by failing their granular web grounding checks at promotion time.

These findings confirm that while individual LLM gates possess distinct operational blind spots, their serial configuration provides an airtight contamination barrier. Evaluation additionally revealed two previously unidentified Critic failure modes — threshold insensitivity to personal queries and output parse failures — underscoring the necessity of layered validation beyond single-gate LLM self-evaluation.

E. RQ3: Evolution Quality Trace and Architectural Ablation

To evaluate factual integrity of AKMA’s knowledge evolution trace, we conduct an architectural ablation study contrasting the full multi-stage framework against a configuration with the Lazy Delta Verification Gate disabled. Traditional non-parametric stores lack autonomous gating and effectively operate at a 100% acceptance rate, making them susceptible to linear quality degradation.

Figure 2 details the impact of gate removal across the original 20-query controlled evaluation suite (Tests A, B, and D, first session).



Fig. 2. Knowledge Base Resection and Contamination Rate

The ablation reveals three key behaviors: (1) **Factual Filtering (Test B)**: full activation restricts promotion to 40.0% of valid refinement candidates, rising to 66.67% in extended evaluation, with unverified mutations safely quarantined rather than committed; (2) **Consensus Sub-Efficacy**: disabling the Lazy Delta gate allows 2 adversarial inputs and 3 unverified mutations to bypass consensus, yielding a 31.3% contamination rate — the consensus layers evaluate semantic alignment, not external factual grounding; (3) **Thematic Deduplication (Test C)**: semantically equivalent but lexically distinct queries across sessions are correctly mapped to the same staging node (similarity 0.9720) rather than creating redundant entries.

By maintaining all 6 gates in sequence, AKMA drops the production contamination rate from 40.0% to 0.0%, directly demonstrating that a closed-loop dual-pool architecture produces a cleaner and more factually resilient knowledge evolution trace than static RAG baselines.

F. RQ4: Computational Overhead and Latency Profiles

Execution latency was profiled across the complete Session 2 evaluation sequence (Figure 3), exposing a runtime profile that correlates directly with network I/O demands and pipeline verification depth.

Queries that fail cross-session candidate matching execute a single-pass ingestion path (5.19 s), while candidates entering full consensus promotion incur the cumulative cost of LLM inference, claim extraction, and sequential Tavily web verification. The primary bottleneck is the Lazy Delta Verification Gate: each unverifiable claim must complete a full network round-trip before the any-fail block policy applies, causing latency to scale with unverified claim count and peak beyond 37 s on attribution-heavy candidates.

The mean consensus latency of 20.99 s reflects the core architectural trade-off: *throughput is sacrificed to guarantee absolute data integrity at the persistence boundary*. This footprint is unsuitable for synchronous query-response interfaces but appropriate for asynchronous background knowledge curation where correctness is prioritized over speed. Future work will address this through parallelized claim verification, cached grounding results, and adaptive claim extraction scoped to high-specificity assertions.

V. LIMITATIONS AND FUTURE WORK

A. Current Framework Limitations

Despite the layered validation mechanisms implemented within AKMA, several systemic boundaries constrain the current minimum viable architecture.

- **Single-Model Monoculture**: All major reasoning operations within the architecture, including routing, refinement, adversarial evaluation, and claim verification, rely on the same underlying language model family. Consequently, systematic model biases or reasoning failures may propagate across multiple validation stages, reducing the independent efficacy of the Critic gate.

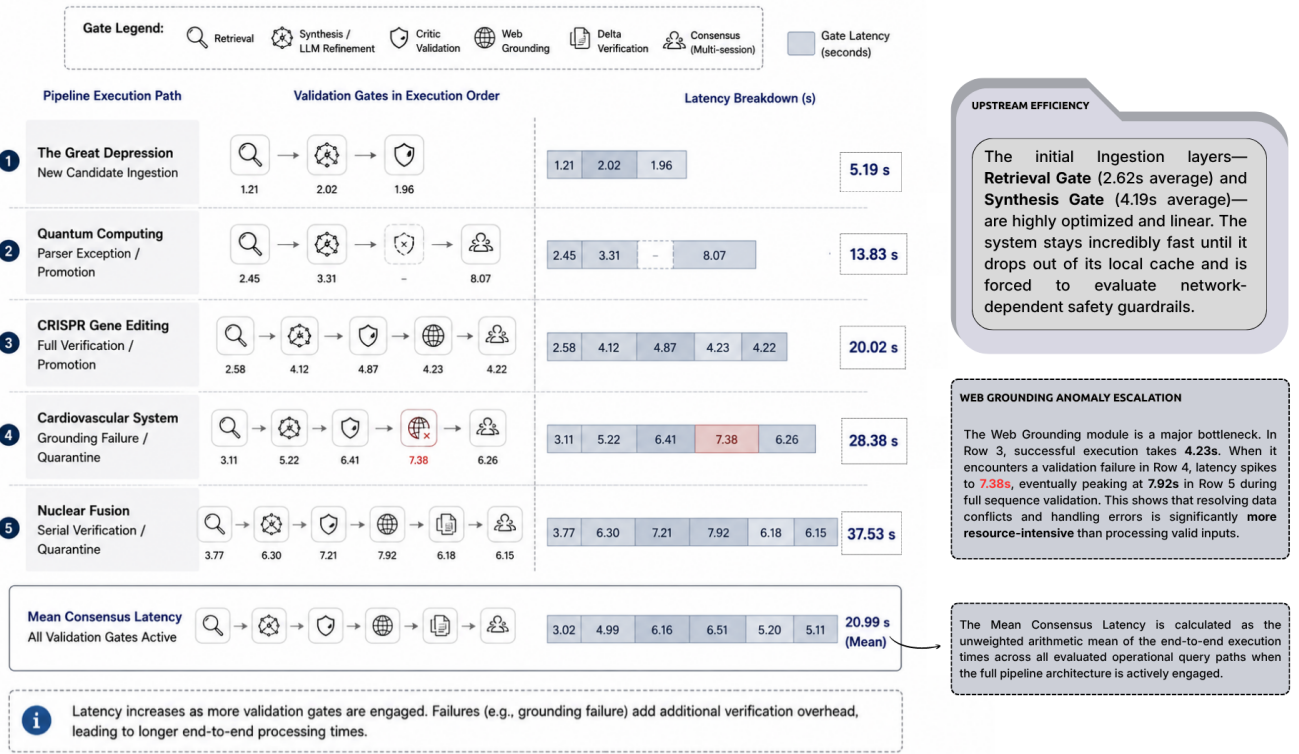


Fig. 3. Validation Gate Contribution to Pipeline Latency

- **Absence of Cross-Document Contradiction Detection:** Candidate mutations are evaluated against their parent documents in isolation and are never cross-referenced against the broader active knowledge corpus. As the knowledge base expands, semantically related documents may evolve independently and introduce conflicting assertions without triggering any global consistency checks.
- **Rigid Claim Extraction Cap:** The strict constraint of `MAX_CLAIMS_TO_CHECK = 3` utilized during the Lazy Delta Verification gate creates a structural bias toward highly compact mutations. Large, multi-fact refinements are heavily penalized or blocked despite being factually correct, limiting overall promotion throughput for complex topics.
- **Critic Gate Robustness:** Two distinct Critic failure modes were observed during evaluation. First, the synthesis scoring rubric does not reliably reject low-specificity personal queries — a personal preference query received scores of 0.70 and 0.90 across two sessions despite containing no web-verifiable content, passing the staging threshold in both cases. Second, Critic output parse failures occurred in two independent traces, triggering the raw promotion fallback and bypassing the refinement quality check. While downstream gates compensated in both cases, these failure modes identify Critic prompt design and output schema enforcement as priority areas for hardening.

Future work will address these limitations through these primary directions: (1) a **heterogeneous multi-model judge panel** utilizing an adversarial ensemble framework and struc-

tured output schemas to eliminate single-model cognitive biases and prompt-parsing failures; (2) a **cross-document consistency graph** that maps the corpus semantically to execute multi-node dependency checks and block logical or statistical contradictions; (3) an **adaptive claim verification mechanism** that dynamically scales verified assertions based on document complexity; and (4) an **upstream privacy and alignment layer** featuring dedicated personal information pre-filters and runtime retries to mitigate threshold sensitivity and parsing vulnerabilities.

VI. CONCLUSION

This paper presented the Autonomous Knowledge Mutation Architecture (AKMA), a self-evolving Retrieval-Augmented Generation framework designed to transform static non-parametric knowledge stores into continuously improving memory systems. Unlike conventional RAG pipelines that treat retrieved documents as immutable references, AKMA introduces a structured mutation lifecycle consisting of candidate staging, adversarial validation, multi-session consensus, and web-grounded verification before permanent knowledge promotion.

Experimental results demonstrated that AKMA can autonomously expand its knowledge base without manual document authoring while maintaining safeguards against semantic drift, hallucinated refinements, and coordinated poisoning attempts. The dual-collection architecture successfully isolated unverified mutations from production knowledge, while the

Critic framework and consensus engine provided multiple layers of protection before promotion.

Despite these contributions, several limitations remain, including the dependence on a single LLM family for evaluation, the absence of cross-document contradiction detection, a rigid claim extraction cap, and identified Critic gate robustness gaps in personal information filtering and output parse reliability. Future work will explore heterogeneous multi-model evaluation, a cross-document consistency graph, an adaptive claim verification mechanism, and an upstream privacy and alignment layer to further improve robustness and scalability.

In general, AKMA demonstrates that persistent knowledge bases can evolve through auditable and consensus-gated mutation while maintaining safeguards against common failure modes.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-rag: Learning to retrieve, generate, and critique through self-reflection,” 2023.
- [3] L. Teng, W. Ni, L. Song, J. Shi, and Y. Li, “Towards self-evolving agents: A dual-process framework for continual context refinement,” *Electronics*, vol. 15, p. 1232, 2026.
- [4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “Memgpt: Towards llms as operating systems,” 2024.
- [5] D. P. Bal and S. Puhon, “Benchmarking retrieval strategies for biomedical retrieval-augmented generation: A controlled empirical study,” 2026.
- [6] J. Wang, X. Yi, C. Xie, *et al.*, “Milvus: A purpose-built vector data management system,” in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pp. 2614–2627, ACM, 2021.
- [7] J. Johnson, M. Matthes, L. Chazeot, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [8] J. J. Pan, J. Wang, and G. Li, “Survey of vector database management systems,” *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, 2024.
- [9] Hugging Face Community, “Sentence-transformers all-minilm-l6-v2 model card,” <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, 2021. Accessed: 2026-05-25.
- [10] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 11 2019.